

Sample Questions

Q1. Implement a CNN feature-extraction stage with convolution, ReLU, and max pooling using NumPy.

A grayscale image is processed by one convolution kernel followed by ReLU activation and 2×2 max pooling. Use the CNN cross-correlation convention, so the kernel must not be flipped.

For an input image of size $H \times W$ and a kernel of size $KH \times KW$, use stride 1 and padding 0 for convolution. The convolution output dimensions are $OH = H - KH + 1$ and $OW = W - KW + 1$.

For each valid image patch, compute $\text{response} = \text{sum}(\text{patch} * \text{kernel}) + \text{bias}$, then apply $\text{ReLU}(\text{response}) = \max(0, \text{response})$.

Apply 2×2 max pooling to the ReLU feature map using stride 2 and no padding. Ignore any final incomplete pooling region.

Do not use convolution, pooling, or neural-network functions from deep-learning libraries.

Input Format

```
H W
H rows of image values
KH KW
KH rows of kernel values
bias
```

Output Format

Print the complete ReLU feature map first, followed by the max-pooled feature map.

Print every floating-point value to exactly four decimal places.

Constraints

- $3 \leq H, W \leq 12$
- $1 \leq KH \leq H$
- $1 \leq KW \leq W$
- $OH \geq 2$ and $OW \geq 2$
- Convolution stride = 1 and padding = 0
- Pooling window = 2×2 with stride 2
- Use NumPy only

Head

```
import numpy as np
```

Body

```
def cnn_feature_extraction(image, kernel, bias):  
    """  
  
    Return:  
        relu_feature_map,  
        pooled_feature_map  
    """  
  
    # Write your complete implementation here  
    pass
```

Tail

```
if __name__ == "__main__":  
    H, W = map(int, input().split())  
    image = np.array([  
        list(map(float, input().split()))  
        for _ in range(H)  
    ], dtype=float)  
  
    KH, KW = map(int, input().split())  
    kernel = np.array([  
        list(map(float, input().split()))  
        for _ in range(KH)  
    ], dtype=float)  
  
    bias = float(input())  
  
    feature_map, pooled_map = cnn_feature_extraction(  
        image, kernel, bias  
    )  
  
    for row in feature_map:  
        print(" ".join(  
            f"{value:.4f}" for value in row  
        ))  
  
    for row in pooled_map:  
        print(" ".join(  
            f"{value:.4f}" for value in row  
        ))
```

Q2. Implement masked scaled dot-product self-attention from scratch using NumPy.

A sequence contains n tokens, each represented using d features. You are given the input matrix X , query, key, and value projection matrices WQ , WK , and WV , and an $n \times n$ binary attention mask.

Generate: $Q = X @ WQ$, $K = X @ WK$, $V = X @ WV$

The scaled attention-score matrix is: $(Q @ K.T) / \text{sqrt}(d)$

The supplied mask determines which token pairs may participate in attention:

- 1 means that the position is allowed.
- 0 means that the position is blocked and must receive attention probability 0.

Every mask row contains at least one allowed position. Apply a numerically stable row-wise Softmax over only the allowed positions and generate the final contextual representation from the value matrix.

Output the complete attention-weight matrix followed by the contextual representation matrix. Do not use built-in attention implementations.

Input Format

```
n d
X rows
d rows of WQ
d rows of WK
d rows of WV
n rows of attention mask
```

Output Format

Print:

- the $n \times n$ attention-weight matrix;
- the $n \times d$ contextual-output matrix.

All floating-point values must be printed to exactly four decimal places.

Constraints

- $1 \leq n \leq 10$
- $1 \leq d \leq 10$
- Each mask value is either 0 or 1.
- Every mask row contains at least one 1.
- Scaling must use $\text{sqrt}(d)$.
- Blocked positions must have probability exactly 0.

Head

```
import numpy as np
```

Body

```
def masked_self_attention(
    X,
    WQ,
    WK,
    WV,
    mask
):
    """
    Return:
        attention_weights,
        contextual_output
```

```
"""
```

```
# Write your complete implementation here
```

```
Pass
```

Tail

```
if __name__ == "__main__":
    n, d = map(int, input().split())

    X = np.array([
        list(map(float, input().split()))
        for _ in range(n)
    ])

    WQ = np.array([
        list(map(float, input().split()))
        for _ in range(d)
    ])

    WK = np.array([
        list(map(float, input().split()))
        for _ in range(d)
    ])

    WV = np.array([
        list(map(float, input().split()))
        for _ in range(d)
    ])

    mask = np.array([
        list(map(int, input().split()))
        for _ in range(n)
    ])

    weights, output = masked_self_attention(
        X,
        WQ,
        WK,
        WV,
        mask
    )

    for row in weights:
        print(" ".join(
            f"{value:.4f}" for value in row
        ))
```

```
for row in output:
    print(" ".join(
        f"{value:.4f}" for value in row
    ))
```

Q3. Implement the embedding-retrieval stage of a Retrieval-Augmented Generation pipeline using NumPy.

You are given one query embedding and m document embeddings, each of dimension d . Retrieve the top- k documents using cosine similarity and construct a weighted context vector from the selected document embeddings.

For every document i , compute cosine similarity as $\text{sim}[i] = \text{dot}(\text{query}, \text{doc}[i]) / (\|\text{query}\| * \|\text{doc}[i]\|)$.

Select the k documents with the highest similarity scores. If two documents have exactly the same similarity, the document with the smaller original zero-based index must rank first.

Apply a numerically stable Softmax only to the k selected similarity scores to obtain retrieval weights. Construct the context vector as the weighted sum of the selected document embeddings in ranking order.

Input Format

```
m d k
query embedding
m rows of document embeddings
```

Output Format

Print, in order:

- all m cosine similarities in original document order;
- the k selected zero-based document indices in ranking order;
- the k retrieval weights in ranking order;
- the d -dimensional weighted context vector.

Print floating-point values to exactly four decimal places.

Constraints

- $1 \leq m \leq 50$
- $1 \leq d \leq 20$
- $1 \leq k \leq m$
- The query embedding is non-zero
- Every document embedding is non-zero
- Use NumPy only

Head

```
import numpy as np
```

Body

```
def retrieve_context(query, documents, k):  
    """  
  
    Return:  
        similarities,  
        selected_indices,  
        retrieval_weights,  
        context_vector  
    """  
  
    # Write your complete implementation here  
    pass
```

Tail

```
if __name__ == "__main__":  
    m, d, k = map(int, input().split())  
  
    query = np.array(  
        list(map(float, input().split())),  
        dtype=float  
    )  
  
    documents = np.array(  
        list(map(float, input().split()))  
        for _ in range(m)  
    ], dtype=float)  
  
    (  
        similarities,  
        selected_indices,  
        retrieval_weights,  
        context_vector  
    ) = retrieve_context(query, documents, k)  
  
    print(" ".join(  
        f"{value:.4f}" for value in similarities  
    ))  
    print(" ".join(map(str, selected_indices)))  
    print(" ".join(  
        f"{value:.4f}" for value in retrieval_weights  
    ))  
    print(" ".join(  
        f"{value:.4f}" for value in context_vector
```

))

Q4. Implement a perceptron classifier for multiple input samples using a supplied threshold.

Unlike a standard perceptron that uses zero as its decision boundary, this classifier uses a supplied threshold.

For each input sample $X = [x_1, x_2, \dots, x_d]$ with weights $W = [w_1, w_2, \dots, w_d]$ and bias b , calculate the score as:

$$\text{score} = (x_1 * w_1) + (x_2 * w_2) + \dots + (x_d * w_d) + b$$

The predicted class is 1 if $\text{score} \geq \text{threshold}$ and 0 otherwise.

Complete the functions in the boilerplate so that the classifier correctly processes all input samples.

Input Format

```
n d
x11 x12 ... x1d
x21 x22 ... x2d
...
xn1 xn2 ... xnd
w1 w2 ... wd
bias
threshold
```

Where:

- n is the number of input samples.
- d is the number of features in each sample.
- The next n lines contain the input samples.
- The following line contains the d weights.
- The next line contains the bias.
- The final line contains the threshold.

Output Format

Print the predicted classes separated by spaces.

```
p1 p2 ... pn
```

Constraints

- $1 \leq n \leq 100$
- $1 \leq d \leq 20$
- Inputs, weights, bias, and threshold may contain decimal values.
- A score exactly equal to the threshold must be classified as 1.

Head

```
import numpy as np
```

Body

```
def weighted_products(sample, weights):
    # Return element-wise weighted products
    pass
```

```
def linear_score(products, bias):
    # Return weighted sum after adding bias
    pass
```

```
def threshold_activation(score, threshold):
    # Return the predicted class
```

```
pass
```

```
def predict_one(sample, weights, bias, threshold):  
    # Predict one sample  
    pass
```

```
def predict_batch(X, weights, bias, threshold):  
    # Predict all samples  
    Pass
```

Tail

```
if __name__ == "__main__":  
    n, d = map(int, input().split())  
  
    X = np.array([  
        list(map(float, input().split()))  
        for _ in range(n)  
    ])  
  
    weights = np.array(  
        list(map(float, input().split()))  
    )  
  
    bias = float(input())  
    threshold = float(input())  
  
    predictions = predict_batch(  
        X, weights, bias, threshold  
    )  
  
    print(" ".join(map(str, predictions)))
```

Q5. Implement a positional-encoding and self-attention sublayer of a Transformer encoder using NumPy.

A sequence contains n token embeddings of dimension d . First generate sinusoidal positional encodings and add them to the input embeddings.

For position pos and integer $i \geq 0$, use $PE[pos, 2i] = \sin(pos / 10000^{(2i/d)})$ and, when dimension $2i+1$ exists, $PE[pos, 2i+1] = \cos(pos / 10000^{(2i/d)})$.

Let $X_p = X + PE$. Generate $Q = X_p @ W_Q$, $K = X_p @ W_K$, and $V = X_p @ W_V$. Compute scaled dot-product self-attention using $(Q @ K.T) / \sqrt{d}$, followed by a numerically stable row-wise Softmax.

Let A be the contextual output. Apply the residual connection $R = X_p + A$, then perform row-wise layer normalization without learnable scale or shift parameters: $(R - \text{mean}) / \sqrt{\text{variance} + \text{epsilon}}$. Use the population variance across the d features of each row.

Do not use built-in Transformer or attention implementations.

Input Format

```
n d
n rows of X
d rows of WQ
d rows of WK
d rows of WV
epsilon
```

Output Format

Print, in order:

- the $n \times d$ positional-encoding matrix;
- the $n \times n$ attention-weight matrix;
- the $n \times d$ layer-normalized output matrix.

Print all floating-point values to exactly four decimal places.

Constraints

- $1 \leq n \leq 10$
- $2 \leq d \leq 10$
- $0 < \text{epsilon} \leq 0.01$
- Softmax must be numerically stable
- Layer normalization is applied independently to each row
- Use NumPy only

Head

import numpy as np

Body

```
def transformer_encoder_sublayer(
    X,
    WQ,
    WK,
    WV,
    epsilon
):
    """
    Return:
        positional_encoding,
        attention_weights,
        normalized_output
    """
    # Write your complete implementation here
    pass
```

Tail

```
if __name__ == "__main__":
    n, d = map(int, input().split())

    X = np.array([
        list(map(float, input().split()))
        for _ in range(n)
    ], dtype=float)

    WQ = np.array([
        list(map(float, input().split()))
        for _ in range(d)
    ], dtype=float)

    WK = np.array([
        list(map(float, input().split()))
        for _ in range(d)
    ], dtype=float)

    WV = np.array([
        list(map(float, input().split()))
        for _ in range(d)
    ], dtype=float)

    epsilon = float(input())
```

```
positional_encoding, weights, output = (
    transformer_encoder_sublayer(
        X, WQ, WK, WV, epsilon
    )
)
```

```
for row in positional_encoding:
    print(" ".join(
        f"{value:.4f}" for value in row
    ))
```

```
for row in weights:
    print(" ".join(
        f"{value:.4f}" for value in row
    ))
```

```
for row in output:
    print(" ".join(
        f"{value:.4f}" for value in row
    ))
```

Q6. Implement one complete training iteration of a two-layer neural network for multi-class classification using NumPy.

The network receives one input vector containing d features and contains h hidden neurons followed by c output neurons. The hidden layer uses ReLU activation, while the output layer uses Softmax. Training uses cross-entropy loss for the supplied target-class index.

Perform complete forward propagation, calculate the loss, determine the gradients of $W1$, $b1$, $W2$, and $b2$ using backpropagation, and update all parameters using the supplied learning rate.

For ReLU, its derivative must be 1 only when the corresponding hidden pre-activation is strictly greater than zero and 0 otherwise.

After the parameter update, perform another forward pass and return the updated class probabilities and predicted class. If multiple classes have the same maximum probability, select the smaller class index.

Do not use PyTorch, TensorFlow, automatic differentiation, or neural-network libraries.

Input Format

```
d h c
X
h rows of W1
b1
c rows of W2
b2
target
learning_rate
```

Output Format

Print:

```
loss
updated W1 rows
updated b1
updated W2 rows
updated b2
updated probabilities
predicted_class
```

Print floating-point values to exactly four decimal places.

Head

```
import numpy as np
```

Body

```
def train_multiclass(
    X,
    W1,
    b1,
    W2,
    b2,
    target,
    learning_rate
):
    """
    Return:
        loss,
        updated_W1,
        updated_b1,
        updated_W2,
        updated_b2,
        updated_probabilities,
        predicted_class
    """

    # Write your complete implementation here
    Pass
```

Tail

```
if __name__ == "__main__":
    d, h, c = map(int, input().split())

    X = np.array(
        list(map(float, input().split())),
        dtype=float
    )

    W1 = np.array([
        list(map(float, input().split()))
        for _ in range(h)
    ], dtype=float)

    b1 = np.array(
```

```

list(map(float, input().split())),
dtype=float
)

```

```

W2 = np.array([
    list(map(float, input().split()))
    for _ in range(c)
], dtype=float)

```

```

b2 = np.array(
    list(map(float, input().split())),
    dtype=float
)

```

```

target = int(input())
learning_rate = float(input())

```

```

(
    loss,
    W1,
    b1,
    W2,
    b2,
    probabilities,
    prediction
) = train_multiclass(
    X, W1, b1, W2, b2,
    target, learning_rate
)

```

```

print(f"{loss:.4f}")

```

```

for row in W1:
    print(" ".join(
        f"{value:.4f}" for value in row
    ))

```

```

print(" ".join(
    f"{value:.4f}" for value in b1
))

```

```

for row in W2:
    print(" ".join(
        f"{value:.4f}" for value in row
    ))

```

```

print(" ".join(

```

```
        f"{value:.4f}" for value in b2
    ))

print(" ".join(
    f"{value:.4f}"
    for value in probabilities
))

print(prediction)
```

Q7. Implement the patch-embedding input stage of a Vision Transformer for a grayscale image using NumPy.

A grayscale image of size $H \times W$ must be divided into non-overlapping square patches of size $P \times P$. H and W are exact multiples of P . Extract patches in row-major spatial order: left to right across the first patch row, then continue top to bottom. Flatten every patch in row-major order into a vector of length P^2 .

Project each flattened patch using $\text{patch_embedding} = W_{\text{patch}} @ \text{patch} + b_{\text{patch}}$, where W_{patch} has size $e \times P^2$ and b_{patch} has e values.

Prepend the supplied class-token vector to the patch embeddings. Then add the supplied positional-embedding matrix element-wise to the complete sequence.

Do not apply activation, normalization, or attention in this question.

Input Format

```
H W P e
H rows of image values
e rows of W_patch, each containing P^2 values
b_patch
class_token
N + 1 rows of positional embeddings
```

Where $N = (H / P) * (W / P)$.

Output Format

Print the final $(N + 1) \times e$ token-sequence matrix after class-token insertion and positional-embedding addition.

Print all floating-point values to exactly four decimal places.

Constraints

- $2 \leq H, W \leq 16$
- $1 \leq P \leq \min(H, W)$
- H and W are divisible by P
- $1 \leq e \leq 12$
- Patches are extracted without overlap
- Use NumPy only

Head

```
import numpy as np
```

Body

```
def vit_patch_embedding(
    image,
    P,
    W_patch,
    b_patch,
    class_token,
    positional_embeddings
):
    """
    Return:
        final_token_sequence
    """
    # Write your complete implementation here
    pass
```

Tail

```
if __name__ == "__main__":
    H, W, P, e = map(int, input().split())

    image = np.array([
        list(map(float, input().split()))
        for _ in range(H)
    ], dtype=float)

    W_patch = np.array([
        list(map(float, input().split()))
        for _ in range(e)
    ], dtype=float)

    b_patch = np.array(
        list(map(float, input().split())),
        dtype=float
    )

    class_token = np.array(
        list(map(float, input().split())),
        dtype=float
    )

    N = (H // P) * (W // P)
    positional_embeddings = np.array([
```

```

list(map(float, input().split()))
for _ in range(N + 1)
], dtype=float)

output = vit_patch_embedding(
    image, P, W_patch, b_patch,
    class_token, positional_embeddings
)

for row in output:
    print(" ".join(
        f"{value:.4f}" for value in row
    ))

```

Q8. Implement the output-processing pipeline of a multi-class neural network.

The network produces k logits. These logits must first be stabilised, converted into exponential values, normalised into Softmax probabilities, and then used to calculate the cross-entropy loss for the correct class.

For each logit $z[i]$, first stabilise it as: $s[i] = z[i] - \max(z)$

Then compute the exponential value: $e[i] = \exp(s[i])$

The Softmax probability for each class is: $p[i] = e[i] / \sum(e)$

For the target class t , the cross-entropy loss is: $L = -\ln(p[t])$

Complete the five functions provided in the boilerplate.

Input Format

```

k
z1 z2 ... zk
target

```

Where:

- k is the number of classes.
- The second line contains k logits.
- target is the zero-based index of the correct class.

Output Format

First line: Softmax probabilities rounded to exactly four decimal places.

Second line: Cross-entropy loss rounded to exactly four decimal places.

```

p1 p2 ... pk
loss

```

Constraints

- $2 \leq k \leq 20$
- $-10000 \leq z[i] \leq 10000$
- $0 \leq \text{target} < k$
- Use the natural logarithm.
- Print all floating-point outputs to exactly four decimal places.

Head

```
import numpy as np
```

Body

```
def stabilize_logits(logits):
```

```

# Step 1: Subtract the maximum logit
pass

def compute_exponentials(stable_logits):
    # Step 2: Compute exponential values
    pass

def normalize_softmax(exp_values):
    # Step 3: Convert exponential values to probabilities
    pass

def target_probability(probabilities, target):
    # Step 4: Return the target-class probability
    pass

def cross_entropy_loss(probability):
    # Step 5: Compute cross-entropy loss
    pass

```

Tail

```

if __name__ == "__main__":
    k = int(input())

    logits = np.array(
        list(map(float, input().split()))
    )

    target = int(input())

    stable = stabilize_logits(logits)
    exp_values = compute_exponentials(stable)
    probabilities = normalize_softmax(exp_values)

    probability = target_probability(
        probabilities, target
    )

    loss = cross_entropy_loss(probability)

    print(" ".join(
        f"{p:.4f}" for p in probabilities
    ))

```

```
))
```

```
print(f"{loss:.4f}")
```

Q9. Implement one LoRA adaptation step for a frozen linear layer using NumPy.

A frozen linear layer has base weight matrix W of size $d_{\text{out}} \times d_{\text{in}}$. LoRA adds two trainable low-rank matrices A of size $r \times d_{\text{in}}$ and B of size $d_{\text{out}} \times r$.

For input x , compute $z = A @ x$ and $\text{output} = W @ x + (\alpha / r) * (B @ z)$. The base matrix W must never be updated.

Use mean-squared error against the supplied target vector: $\text{loss} = \text{mean}((\text{output} - \text{target})^2)$. Compute the gradients of A and B manually from the same forward pass and update both matrices simultaneously using Gradient Descent.

After the update, perform another forward pass using the updated A and B and print the updated output. Do not use automatic differentiation or deep-learning libraries.

Input Format

```
d_in d_out r
x
d_out rows of W
r rows of A
d_out rows of B
target
alpha
learning_rate
```

Output Format

Print, in order:

```
loss
updated A rows
updated B rows
updated_output
```

Print every floating-point value to exactly four decimal places.

Constraints

- $1 \leq d_{\text{in}}, d_{\text{out}} \leq 10$
- $1 \leq r \leq \min(d_{\text{in}}, d_{\text{out}})$
- $\alpha > 0$
- $0 < \text{learning_rate} \leq 1$
- W remains frozen
- A and B must be updated using gradients from the pre-update forward pass
- Use NumPy only

Head

```
import numpy as np
```

Body

```
def lora_training_step(
    x,
    W,
    A,
    B,
    target,
    alpha,
    learning_rate
):
    """
    Return:
        loss,
        updated_A,
        updated_B,
        updated_output
    """
    # Write your complete implementation here
    pass
```

Tail

```
if __name__ == "__main__":
    d_in, d_out, r = map(int, input().split())

    x = np.array(
        list(map(float, input().split())),
        dtype=float
    )

    W = np.array([
        list(map(float, input().split()))
        for _ in range(d_out)
    ], dtype=float)

    A = np.array([
        list(map(float, input().split()))
        for _ in range(r)
    ], dtype=float)

    B = np.array([
        list(map(float, input().split()))
        for _ in range(d_out)
```

```

], dtype=float)

target = np.array(
    list(map(float, input().split())),
    dtype=float
)

alpha = float(input())
learning_rate = float(input())

loss, A, B, output = lora_training_step(
    x, W, A, B, target, alpha, learning_rate
)

print(f"{loss:.4f}")
for row in A:
    print(" ".join(
        f"{value:.4f}" for value in row
    ))
for row in B:
    print(" ".join(
        f"{value:.4f}" for value in row
    ))
print(" ".join(
    f"{value:.4f}" for value in output
))

```

Q10. Implement a two-head self-attention mechanism from scratch using NumPy.

The input contains n token representations of dimension d , where d is always even. Query, key, and value representations are generated using the supplied $d \times d$ projection matrices WQ , WK , and WV .

The projected representations must be divided equally into two attention heads, each having dimension $d/2$. Each head independently calculates scaled dot-product attention using its own portion of the query, key, and value representations. Scaling must use the square root of the head dimension.

Softmax must operate row-wise and must be implemented in a numerically stable manner.

The contextual representations produced by the two heads are concatenated in their original head order and transformed using the supplied output projection matrix WO .

Your program must output:

- attention-weight matrix of Head 1;
- attention-weight matrix of Head 2;
- final contextual representation after concatenation and output projection.

Do not use any built-in multi-head-attention implementation.

Input Format

```

n d
X rows
d rows of WQ
d rows of WK
d rows of WV
d rows of WO

```

Output Format

Print the $n \times n$ Head-1 attention matrix, followed by the $n \times n$ Head-2 attention matrix, followed by the final $n \times d$ output matrix.
Print every floating-point value to exactly four decimal places.

Constraints

- $1 \leq n \leq 10$
- $2 \leq d \leq 10$
- d is even.
- Each head has dimension $d/2$.
- Softmax operates independently for every row of every head.
- Scaling uses $\sqrt{d/2}$.

Head

```
import numpy as np
```

Body

```
def two_head_self_attention(
    X,
    WQ,
    WK,
    WV,
    WO
):
    """
    Return:
        head1_weights,
        head2_weights,
        final_output
    """

    # Write your complete implementation here

    Pass
```

Tail

```
if __name__ == "__main__":
    n, d = map(int, input().split())

    X = np.array([
        list(map(float, input().split()))
        for _ in range(n)
    ], dtype=float)

    WQ = np.array([
        list(map(float, input().split()))
        for _ in range(d)
    ], dtype=float)
```

```

WK = np.array([
    list(map(float, input().split()))
    for _ in range(d)
], dtype=float)

```

```

WV = np.array([
    list(map(float, input().split()))
    for _ in range(d)
], dtype=float)

```

```

WO = np.array([
    list(map(float, input().split()))
    for _ in range(d)
], dtype=float)

```

```

head1, head2, output = (
    two_head_self_attention(
        X, WQ, WK, WV, WO
    )
)

```

```

for row in head1:
    print(" ".join(
        f"{value:.4f}" for value in row
    ))

```

```

for row in head2:
    print(" ".join(
        f"{value:.4f}" for value in row
    ))

```

```

for row in output:
    print(" ".join(
        f"{value:.4f}" for value in row
    ))

```

Q11. Implement Intersection over Union and greedy Non-Maximum Suppression for object-detection boxes using NumPy.

You are given n axis-aligned bounding boxes with confidence scores. Each box is represented as $(x1, y1, x2, y2, score)$, where $x2 > x1$ and $y2 > y1$.

For two boxes, compute intersection area using the overlap of their coordinate ranges. Compute each box area as $(x2 - x1) * (y2 - y1)$, without adding 1 to any coordinate difference. Then compute $IoU = \text{intersection} / \text{union}$.

Perform greedy Non-Maximum Suppression. Sort boxes by descending score, breaking equal-score ties using the smaller original zero-based index. Select the highest-ranked remaining box, then suppress every lower-ranked remaining box whose IoU with the selected box is strictly greater than the supplied threshold. Continue until no boxes remain.

Input Format

```
n
x1 y1 x2 y2 score    (repeated for n boxes)
iou_threshold
```

Output Format

First line: zero-based indices of all kept boxes in the exact order they are selected by NMS.

Second line: confidence scores of the kept boxes in the same order, printed to exactly four decimal places.

Constraints

- $1 \leq n \leq 100$
- $0 \leq iou_threshold \leq 1$
- $x2 > x1$ and $y2 > y1$ for every box
- Confidence scores may be any finite decimal values
- Suppress only when $IoU > iou_threshold$
- Use NumPy only

Head

```
import numpy as np
```

Body

```
def compute_iou(box_a, box_b):
    # Return the Intersection over Union of two boxes
    pass

def greedy_nms(boxes, iou_threshold):
    """
    Return:
        kept_indices,
        kept_scores
    """
    # Write your complete NMS implementation here
    pass
```

Tail

```
if __name__ == "__main__":
    n = int(input())

    boxes = np.array([
        list(map(float, input().split()))
        for _ in range(n)
    ], dtype=float)

    iou_threshold = float(input())

    kept_indices, kept_scores = greedy_nms(
        boxes, iou_threshold
    )

    print(" ".join(map(str, kept_indices)))
    print(" ".join(
        f"{score:.4f}" for score in kept_scores
    ))
```

Q12. Implement one complete training step of a two-layer neural network for binary classification using NumPy.

The network contains:

- d input features
- h neurons in one hidden layer
- ReLU activation in the hidden layer
- One Sigmoid output neuron
- Binary Cross-Entropy loss
- Gradient Descent for parameter updates

You must implement the complete computation manually. Do not use PyTorch, TensorFlow, automatic differentiation, or neural-network libraries.

Input Format

d h

```

x1 x2 ... xd
W1 row 1
W1 row 2
...
W1 row h
b11 b12 ... b1h
w21 w22 ... w2h
b2
y
learning_rate

```

Where:

- d = number of input features
- h = number of hidden neurons
- X contains d input values
- $W1$ is an $h \times d$ matrix
- $b1$ contains h hidden-layer biases
- $W2$ contains h output-layer weights
- $b2$ is the output-layer bias
- y is either 0 or 1
- `learning_rate` is the Gradient Descent learning rate

Output Format

Print the following in order:

```

loss
updated W1 row 1
updated W1 row 2
...
updated W1 row h
updated b1
updated W2
updated b2
updated_probability
predicted_class

```

Print all floating-point values to exactly four decimal places.

Constraints

- $1 \leq d \leq 10$
- $1 \leq h \leq 10$
- y is either 0 or 1
- $0 < \text{learning_rate} \leq 1$
- Inputs, weights, and biases may be positive, negative, or zero
- Use NumPy only
- Do not use automatic differentiation
- Use ReLU derivative 0 when the pre-activation is exactly zero

Head

```
import numpy as np
```

Body

```
def train_one_step(X, W1, b1, W2, b2, y, learning_rate):
    """
```

Perform one complete neural-network training step.

Return:

```

loss,
updated_W1,
updated_b1,
updated_W2,
updated_b2,
updated_probability,
predicted_class
    """

```

Write your complete solution here

```
pass
```

Tail

```
if __name__ == "__main__":
    d, h = map(int, input().split())

    X = np.array(
        list(map(float, input().split())),
        dtype=float
    )

    W1 = np.array([
        list(map(float, input().split()))
        for _ in range(h)
    ], dtype=float)

    b1 = np.array(
        list(map(float, input().split())),
        dtype=float
    )

    W2 = np.array(
        list(map(float, input().split())),
        dtype=float
    )

    b2 = float(input())
    y = int(input())
    learning_rate = float(input())

    (
        loss,
        W1,
        b1,
        W2,
        b2,
        probability,
        prediction
    ) = train_one_step(
        X, W1, b1, W2, b2,
        y, learning_rate
    )

    print(f"{loss:.4f}")

    for row in W1:
        print(" ".join(
            f"{value:.4f}" for value in row
        ))

    print(" ".join(
```

```
    f"{value:.4f}" for value in b1
))

print(" ".join(
    f"{value:.4f}" for value in W2
))

print(f"{b2:.4f}")
print(f"{probability:.4f}")
print(prediction)
```